

Count Triplets Smaller Than Target

Two approaches, visualized execution, and the core logic to remember

Brute Force · $O(n^3)$

Two Pointers · $O(n^2)$

Reference notes · CountThreeSumSmallerThanTarget.java

Problem Statement

Given an array `arr[]` of distinct integers and a value `sum`, count the triplets `(i, j, k)` with `i < j < k` such that `arr[i] + arr[j] + arr[k] < sum`.

`arr =`
`[5,1,3,4,7]`
`sum = 12`

→

count triplets
summing below target

→

Output: 4

Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n^3)$	$O(1)$	Check every triplet directly
Two Pointers	$O(n^2)$	$O(1)$	Sorted array lets you count a whole range of valid triplets in one step

Big picture: This is a counting variant of 3Sum. Instead of collecting or comparing individual triplets, the sorted two-pointer walk lets you count a *whole batch* of valid triplets in one move — whenever `sum < target`, every index between `left` and `right` also works with the current `left`.

1 · Brute Force

$O(n^3)$ time

$O(1)$ space

Try every
(i, j, k)



sum < target?



yes →
count++

```
// check every triplet, count the ones under target
private static int bruteForce(int[] a, int targetSum) {
    int count = 0;

    for (int i = 0; i < a.length - 2; i++) {
        for (int j = i + 1; j < a.length - 1; j++) {
            for (int k = j + 1; k < a.length; k++) {
                if (a[i] + a[j] + a[k] < targetSum) {
                    count++;
                }
            }
        }
    }

    return count;
}
```

Recall trick: "Check every triplet, count the ones under target." Correct and simple, but $O(n^3)$ — too slow once `arr.length` grows.

2 · Two Pointers

O(n²) time

O(1) space

Sort array,
fix a[i]



left = i+1
right = n-1



sum < target?
count += right-left



too small→left++
too big→right--

```
// sum too small: everything between left and right also works with this left
private static int usingTwoPointers(int a[], int targetSum) {
    Arrays.sort(a);
    int count = 0;

    for (int i = 0; i < a.length - 2; i++) {
        int left = i + 1;
        int right = a.length - 1;

        while (left < right) {
            int sum = a[i] + a[left] + a[right];

            if (sum < targetSum) {
                count = count + (right - left);
                left++;
            } else {
                right--;
            }
        }
    }

    return count;
}
```

Trace for a = [5,1,3,4,7] → sorted [1,3,4,5,7], targetSum = 12:

i	a[i]	left	right	sum	< target?	action	count after
0	1	1	4	11	yes	count += (4-1)=3, left++	3
0	1	2	4	12	no	right--	3
0	1	2	3	10	yes	count += (3-2)=1, left++	4
1	3	2	4	14	no	right--	4
1	3	2	3	12	no	right-- → left<right false	4
2	4	3	4	16	no	right-- → done	4

Recall trick: "Sum too small → every element between left and right also works with this left → count them all at once, then move left." This "count in bulk" idea turns an O(n) inner pair-search into an O(n) counting pass without checking every pair individually — because the array is sorted, if $a[i] + a[\text{left}] + a[\text{right}] < \text{target}$, replacing right with anything smaller (down to left+1) still keeps the sum below target.

TL;DR — For Future Me

What to remember

- **Best solution:** Two pointers, O(n²) — "if sum < target, everything between left and right also works with this left — count them all at once (right - left), then advance left."
- **Brute force** is O(n³) — correct but only fine for small inputs.
- **Underlying pattern:** a counting variant of 3Sum / 3Sum Closest — instead of finding exact matches, it exploits sorted order to count a whole batch of valid triplets per pointer move. This reappears whenever the question is "how many pairs/triplets satisfy an inequality" rather than "find the exact ones."